

C++ Distributed Counters

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0261R4
Date:	2020-01-12
Audience	LEWG
Revises:	P0261R3
Author:	Lawrence Cowl
Contact	Lawrence@Cowl.org

Abstract

We propose four class templates implementing distributed atomic counters.

Contents

[Introduction](#)

[Solution](#)

[Distributable Counter](#)

[Counter Broker](#)

[Counter Array](#)

[Implementation](#)

[Questions, Answers and Alternatives](#)

[Wording](#)

[?? Distributed counters \[distctr\]](#)

[??1 General \[distctr.general\]](#)

[??1.1 Synchronization \[distctr.sync\]](#)

[??1.2 Value Computation \[distctr.value\]](#)

[??2 Synopsis \[distctr.syn\]](#)

[??4 Bumper concept \[distctr.bumper\]](#)

[??5 Class template distributable_counter \[distctr.dstctr\]](#)

[??6 Class template counter_broker \[distctr.broker\]](#)

[??7 Class distributable_counter_array \[distctr.generalarray\]](#)

[??8 Class counter_broker_array \[distctr.brokerarray\]](#)

[Revision History](#)

[References](#)

Introduction

Counters are ubiquitous in computing. For multi-threaded programs, atomic integers can provide safe concurrent counting.

In many cases, counters are much more commonly incremented than read. For example, long-running multithreaded server programs may maintain many counters to aid in program diagnosis. Counters are often read only in response to a network query or program termination.

Unfortunately, the cost incrementing an atomic integer is significantly greater than reading one, even when reads are rare relative to increments. As the program scales, the cost of incrementing rarely-read atomic integers can limit program performance.

We need a mechanism that minimizes the cost of incrementing a counter, even if it has increased costs to read the counter.

Cilk adder-reducers [\[Cilk-Reducer\]](#) are one such mechanism. Unfortunately, the reduction is control-dependent and hence not suitable for external inspection of the count.

We propose a counter that has a complexity similar to atomic integers in the simple case, but enables programmers to easily distribute that counter across multiple threads when scalability becomes a concern.

This proposal restricts itself to precise counters. Statistical counters, where the read count may be slightly different from the count operations, can produce better performance [\[Dice-Lev-Moir-2013\]](#).

Solution

We propose two coordinated counter types that collectively provide for distributed counting. The essential component of the design is a trade-off between the cost of incrementing the counter, the cost of loading (reading) the counter value, and the "lag" between an increment and its appearance in a "read" value. That is, the read of a counter value may not reflect the most recent increments. However, no count will be lost.

These counters are parameterized by the base integer type that maintains the count. Avoid situations that overflow the integer, as that may have uninterpretable or undefined behavior. This constraint implies that counters must be sized to their use. (For dynamic mitigation, see the `exchange` operation below.)

The base of the design is the concept of a counter bumper. This concept only provides for increasing or decreasing a counter. None of its operations provide any means to read the count.

Distibutable Counter

The first type programmers will encounter is the `distributable_counter`. This counter implements the bumper concept and provides two additional operations, `load` and `exchange`. The `load` operation returns the current count. Note that 'current' is a fuzzy notion when other threads are concurrently incrementing the counter. The `exchange` operation replaces the current count with the given value and returns the old value. This operation has particular utility in long-running programs in occasionally draining the counter to prevent integer overflow.

```
distributable_counter<int> red_count;

void count_red( Bag bag ) {
    for ( auto i: bag )
        if ( i.is_red() )
            ++red_count;
}

int how_many_red() {
    return red_count.load();
}
```

Counter Broker

When contention becomes a problem, programmers distribute the counting with a `counter_broker` objects. These will typically be thread-local objects, but could be CPU-local objects or even stack-based objects.

```
distributable_counter<int> red_count;
thread_local counter_broker<int> thread_red( red_count );

void count_red( Bag bag )
    for ( auto i: bag )
        if ( i.is_red() )
            ++thread_red;
}
```

The association of counter brokers to general counter variables is an essential part of the distribution of counters, and not easily moved or copied. Therefore, we provide no copy or move operations for these types.

Counter Array

Managing the association between general counters and their attached brokers can require a non-trivial amount of space and time. Therefore, we provide a means to handle many counters with one name with the `distributable_counter_array`. The size of the counter arrays is fixed at construction time. Counter arrays have the same structure as single-value counters, with the following exceptions.

- Increment a counter by first indexing into its array and then applying one of the bumper operations. E.g. `++ctr_ary[i]`.
- The `load` and `exchange` operations take an additional index parameter.

Implementation

Olivier Giroux suggested an implementation in which the set of attached brokers was tracked by an intrusive list through the brokers themselves, rather than through a separate set data structure. This approach has some advantages.

- No dynamic allocation is required.
- The constructor of the general counter can be `constexpr`, which avoids initialization order problems for global objects.
- The head of the list is null until such time as a broker attaches itself, which substantially reduces the cost of using a general counter in the case where there are no brokers.

The benefit of a `constexpr` initializer is high enough so that we choose to adopt the `constexpr` initializer. Unfortunately, counter arrays require allocation for their array of bumpers, and so cannot have `constexpr` initializers.

An implementation of an earlier version of the design (with the separate broker set data structure) is available from the Google Concurrency Library <https://github.com/alasdairmackintosh/google-concurrency-library> at [.../blob/master/include/counter.h](https://github.com/alasdairmackintosh/google-concurrency-library/blob/master/include/counter.h). This paper's `distributable_counter` maps to `counter::strong_duplex` and this paper's `counter_broker` maps to `counter::strong_broker`.

Questions, Answers and Alternatives

Are these counters good for reference counting?

No. The design point for the paper is when reads are rare relative to increments. Reference counting has closer to a one-to-one ratio.

Isn't a `distributable_counter` just a wrapper around an atomic integral?

No. The `distributable_counter` maintains the list of associated brokers and uses that list to poll the brokers for load and exchange operations.

Why is it called a `counter_broker`?

The type brokers the increments for the counter.

Can you come up with better names?

I welcome specific suggestions with rationale.

Aren't the `counter_brokers` a needless complication?

No. The essential performance gain comes from *distribution*, which requires variables in different parts of the system [\[Vollmann-2015\]](#).

Wouldn't automatically creating the `counter_brokers` be better?

There is no language mechanism for automatically creating the distributed variables. Mechanisms do exist on *some* systems for hand-coding a distribution. The standard should not rely on narrowly implemented facilities.

Even so, these hand-coded mechanisms generally do not have the performance of language-defined `thread_local` variables. The cost of finding 'your' broker may quickly become the performance bottleneck. Access to variables is best integrated into the compilation system.

The current design enables end-user programmers to exploit any system-provided distribution mechanism.

When high-performance distribution mechanisms are more widely implemented and integrated into the compilation system, automatic distribution becomes more viable.

Your example shows `thread_local` brokers. Wouldn't CPU-local brokers be better?

Yes. However, not all platforms provide efficient access to a CPU-local variable. If your platform provides CPU-local variables, you can use brokers with them.

A dedicated thread-distributed counter could use `store(load()+1)` to implement increments, right?

Yes. This technique was used in the `weak_duplex` and `weak_broker` types in earlier versions of the paper. The downside is that one cannot have the exchange operation because the exchange would be a concurrent write to the broker. SG1 chose to remove the weaker version to simplify the interface.

Can we use 'restartable sequences'?

Restartable sequences only now have limited availability. More importantly, they require *exclusive* CPU access, or at least the ability to detect potential non-exclusive access, which effectively prohibits the exchange operation. That is, restartable sequences only work with the weaker broker.

Can we get the weaker broker back?

There is nothing in this proposal that precludes adding the weaker version later. Given that SG1 has a preference for a simple initial facility, adding the weaker version should be a separate paper.

Can you use brokers with automatic variables?

Yes. Higher-performance facilities for use with automatic variables were in the earlier version of the paper. They used a 'push' model of returning counts to the main counter to eliminate atomic variable cost entirely. These facilities were removed by SG1 to reduce the complexity of the interface.

Wording

The distributed counter definition is as follows.

?.? Distributed counters [distctr]

Add a new section.

?.?.1 General [distctr.general]

Add a new section.

This section provides mechanisms for distributed counters. These types optimize modifying the count over reading the count. That is, reading is assumed to be relatively rare compared to modifying. These mechanisms ease the production of scalable concurrent programs.

The distributable_counter provides a single counter. When contention on that counter becomes undesirable, one can distribute the counting and representation by associating one or more counter_brokers with the distributable_counter. Together, they provide a single abstract counter.

Every operation on a distributable_counter its associated counter_broker is an operation on the abstract counter.

Likewise, distributable_counter_array and counter_broker_array provides an indexed set of abstract counters. Operations on both with the same index are operations on a single abstract counter.

Each abstract counter has a set M consisting of the modify operations ($+=$, $-=$, and exchange) and a set R consisting of the read operations (load and exchange). The exchange operations exist in both sets.

?.?.1.1 Synchronization [distctr.sync]

Add a new section.

All read and modify operations are atomic. [Note: They will not cause a data race. —end note]

The classes described in [distctr] impose no memory ordering between increment and decrement operations and any other operations. So, the set of modify operations might not be totally ordered. [Note: Detecting a count value that indicates that an event has occurred does not mean that any other memory updates associated with that event are visible to the current thread. Acting on any particular count requires additional synchronization. —end note]

?.?.1.2 Value Computation [distctr.value]

Add a new section.

The set of observed modify operations O_i of a read operation R_i of an abstract counter is the union of the following sets:

- The set P_i contains the union of O_j such that R_j happens before R_i .
- The set B_i contains those operations in M that happen before R_i .
- The set U_i is an arbitrary subset of those operations in M that neither happen after nor happen before R_i and

are not R_i . [*Note: The exchange operation does not observe its own modification. —end note*]

The abstract value V_i of a read operation R_i is the sum of:

- the initial abstract value I ,
- the argument to each $+=$ operation M_i that is in O_i ,
- the negative of the argument to each $-=$ operation M_i that is in O_i , and
- the argument to each exchange operation M_i minus its abstract value V_i such that the operation M_i is in O_i .

The actual value is the abstract value when the abstract value never exceeds the range of the template parameter type. Otherwise, the actual value is implementation-defined.

2.2.2 Synopsis [distctr.syn]

Add a new section.

```
template< typename Integral > class distributable_counter;  
template< typename Integral > class counter_broker;  
template< typename Integral > class distributable_counter_array;  
template< typename Integral > class counter_broker_array;
```

2.2.4 Bumper concept [distctr.bumper]

Add a new section.

The bumper concept provides a common interface for increasing or decreasing the value of a counter. It consists of the following operations. [*Note: These operations specifically return void to avoid leaking information that would defeat the purpose of optimizing for increments over loads. —end note*]

```
void operator +=( integer )  
void operator -=( integer )
```

Effects:

Atomically add/subtract a value to/from the counter.

```
void operator ++()  
void operator ++(int)  
void operator --()  
void operator --(int)
```

Effects:

As if $counter+=1$ or $counter-=1$, respectively.

2.2.5 Class template distributable_counter [distctr.dstctr]

Add a new section.

A distributable_counter counter need not be distributed. It can act as a simple counter.

```
namespace std {  
template< typename Integral >  
class distributable_counter  
{  
    distributable_counter( const distributable_counter& ) = delete;  
    distributable_counter& operator=( const distributable_counter& ) = delete;  
public:  
    explicit constexpr distributable_counter( Integral );  
    constexpr distributable_counter();  
    ~distributable_counter();  
    void operator +=( Integral );  
    void operator -=( Integral );  
};
```

```

    void operator ++();
    void operator ++(int);
    void operator --();
    void operator --(int);
    Integral load();
    Integral exchange( Integral to );
}

```

The `Integral` template type parameter shall be an integral type for which `std::atomic` has a specialization (29.5 [atomics.types.generic]).

The `distributable_counter` class template implements the bumper concept ([\[distctr.bumper\]](#)).

```

explicit constexpr distributable_counter( Integral )
constexpr distributable_counter()

```

Effects:

Initialize the `distributable_counter` with the given value, or zero if no value is given.

```

~distributable_counter()

```

Requirements:

All attached `counter_broker` objects shall have been previously destroyed.

Effects:

Destroys the `distributable_counter`.

```

Integer load()

```

Returns:

Atomically returns the value of the `distributable_counter`. *[Note: The value will be an approximation if any threads may concurrently update the `distributable_counter`. —end note]*

```

Integer exchange( Integer )

```

Effects:

Atomically replaces the value of the `distributable_counter` with the value of the parameter.

Returns:

The replaced value.

??6 Class template `counter_broker` [distctr.broker]

Add a new section.

```

template< typename Integral >
class counter_broker
{
    counter_broker( const counter_broker& ) = delete;
    counter_broker& operator=( const counter_broker& ) = delete;
public:
    counter_broker( distributable_counter<Integral>& );
    ~counter_broker();
    void operator +=( Integral );
    void operator -=( Integral );
    void operator ++();
    void operator ++(int);
    void operator --();
    void operator --(int);
};

```

The `Integral` template type parameter shall be an integral type for which `std::atomic` has a specialization (29.5 [atomics.types.generic]).

The `counter_broker` class template implements the bumper concept ([\[distctr.bumper\]](#)).

counter_broker(distributable_counter<Integral>&)

Effects:

Initialize the broker, atomically associating it with the given distributable_counter.

~counter_broker()

Effects:

Atomically moves any part of the count value retained within the counter_broker to its associated distributable_counter. Atomically disassociates the counter_broker from the distributable_counter. Destroys the counter_broker.

29.7.7 Class distributable_counter_array [distctr.generalarray]

Add a new section.

```
template< typename Integral >
class distributable_counter_array
{
    distributable_counter_array( const distributable_counter_array& ) = delete;
    distributable_counter_array& operator=( const distributable_counter_array& ) = delete;
public:
    typedef size_t size_type;
    distributable_counter_array( size_type size );
    some_bumper_type& operator[]( size_type idx );
    size_type size();
    Integral load( size_type idx );
    Integral exchange( size_type idx, Integral value );
};
```

The Integral template type parameter shall be an integral type for which std::atomic has a specialization (29.5 [atomics.types.generic]).

constexpr distributable_counter_array(size_type);

Effects:

Initialize the distributable_counter_array with the given number of counters, each initialized to zero.

~distributable_counter_array()

Requirements:

All attached counter_broker_array objects shall have been previously destroyed.

Effects:

Destroys the distributable_counter_array.

size_type size()

Returns:

The value passed to the constructor.

some_bumper_type& operator[](size_type index)

Precondition:

$0 \leq \text{index} < \text{size}()$

Returns:

A reference to an object, identified by the index, implementing the bumper concept [\[distctr.bumper\]](#).

Integer load(size_type index)

Precondition:

$0 \leq \text{index} < \text{size}()$

Returns:

the value of the bumper with the given `index`. [Note: The value will be an approximation if any threads may concurrently update the counter. —end note]

Integer exchange(size type index, Integer value)

Precondition:

$0 \leq \text{index} < \text{size}()$

Effects:

Atomically replaces the value of the bumper at the given `index` with the given `value`.

Returns:

The replaced value.

2.8 Class `counter_broker_array` [`distctr.brokerarray`]

Add a new section.

```
template< typename Integral >
class counter_broker_array
{
    counter_broker_array( const counter_broker_array& ) = delete;
    counter_broker_array& operator=( const counter_broker_array& ) = delete;
public:
    typedef size_t size_type;
    counter_broker_array( distributable_counter_array<Integral>& );
    some_bumper_type& operator[]( size_type idx );
    size_type size();
};
```

The `Integral` template type parameter shall be an integral type for which `std::atomic` has a specialization (29.5 [atomics.types.generic]).

constexpr counter_broker_array(distributable_counter_array< Integral >&);

Effects:

Initialize the broker array. Atomically associate the `counter_broker_array` with the `distributable_counter_array`.

~counter_broker_array()

Effects:

Atomically moves each count value retained within the broker to its associated position in the counter array.
Atomically disassociates the broker array from the counter array. Destroys the broker array.

size_type size()

Returns:

The value passed to the constructor.

some_bumper_type& operator[](size_type index)

Precondition:

$0 \leq \text{index} < \text{size}()$

Returns:

A reference to an object, identified by the `index`, implementing the bumper concept [[distctr.bumper](#)].

Revision History

This paper revises [P0261R3 - 2017-03-13](#). Its changes are as follows.

- LEWGI suggested several renamings of `general_counter`, of which the top two were `distributed_counter` and `atomic_counter`, with `shared_counter` as honorable mention. This revision chooses `distributable_counter` with text clarifying that a `distributable_counter` need not be distributed, but may serve as a simple atomic counter.
- Made `distributable_counter::load` const.
- Added `is_lock_free` and `is_always_lock_free`.

P0261R3 revised [P0261R2 - 2017-02-05](#). Its changes are as follows.

- Add introductory text to `[distctr.general]`.
- Define modify and read operations in `[distctr.general]`.
- Add a Synchronization section `[distctr.sync]` in `[distctr.general]`. Define read operations that happen after other read operations to include all those increments seen by the other. Define the increment and decrement operations to impose no memory order.
- Add a Value Computation section `[distctr.value]` in `[distctr.general]`. Define the abstract value of the counter for any read operation as the sum of observed modify operations.
- Add a questions, answers and alternatives section to address design choices.
- Add the descriptions of the `size` function. Add prerequisites using `size` where appropriate.

P0261R2 revised [P0261R1 - 2016-10-13](#). Its changes are as follows.

- Merge all synchronization clauses into a single new section. Require cache coherence on counts.
- Clarify that moving the count from the broker does so an atomic element at a time, not atomically in bulk.
- Remove "There is no default value" as it follows from the declaration and is potentially confusing.
- Move the note about increments returning `void` to the top of the section so that it applies to all operations.

P0261R1 revised [P0261R0 - 2016-02-14](#). Its changes are as follows.

- Clarify that `general_counter::load` is atomic.
- Use the type-specific phrase "`counter_broker` object" rather than the general phrase "counter broker" in the proposed wording. Do the same with the array forms.
- Correct parameter and description of effective broker constructors.
- Add a `size_type` to the counter array classes.
- Clarify initialization of counter array values.
- Clarify the adoption of constexpr `general_counter` constructors.
- Expand on the problem of counter overflow.
- Update the document header.
- Add an abstract.
- Add subheadings to Solution.
- Do general editorial cleanup.

P0261R0 revised [N3706 - 2013-09-01](#). Its changes are as follows:

- Add more context to the introduction and rework subsequent discussion.
- Reduce the number of types to four, a `general_counter` equivalent to the earlier `strong_duplex`, a `counter_broker` equivalent to the earlier `strong_broker`, a `general_counter_array` equivalent to the earlier `strong_duplex_array`, and a `counter_broker_array`

equivalent to the earlier `strong_broker_array`.

- Remove the atomicity specification. Only full and relaxed atomicity is now supported.
- As a consequence of the above two points, drastically reduce the solution presentation.
- Update the implementation discussion with a new technique and new pointers to the code base.
- Add proposed wording.

N3706 revised [N3355 = 12-0045 - 2012-01-14](#). Its changes are as follows:

- Add more context to the introduction.
- Change named `inc` and `dec` functions to operators.
- Place all declarations within namespace `counter`. The namespace serves to avoid redundancy in names.
- Rename types for increased clarity.
- Change separate serial and atomic counter types with a single type templated on the atomicity.
- Add issues of memory order and contention to the atomicity parameter.
- Add counter arrays.
- Add open issues section.
- Add implementation section.
- Add revision history section.
- Add references section.

References

[Cilk-Reducer]

"Reducers", http://software.intel.com/sites/products/documentation/hpc/composerxe/en-us/2011Update/cpp/lin/cref_cls/common/cilk_bk_reducer_intro.htm; "Intel Cilk Plus", http://software.intel.com/sites/products/documentation/hpc/composerxe/en-us/2011Update/cpp/lin/cref_cls/common/cilk_bk_using_cilk.htm; "Intel C++ Compiler XE 12.1 User and Reference Guides", http://software.intel.com/sites/products/documentation/hpc/composerxe/en-us/2011Update/cpp/lin/main/main_cover_title.htm

[Dice-Lev-Moir-2013]

"Scalable Statistics Counters"; Dave Dice, Yossi Lev, Mark Moir; SPAA'13, June 23-25, 2013, Montréal Québec, Canada.

[Lev-Moir-2011]

"Lightweight Parallel Accumulators Using C++ Templates"; Yossi Lev, Mark Moir; ICSE'11, May 21-28, 2011, Waikiki, Honolulu, HI, USA

[Vollmann-2015]

Atomic Counters: A Lesson on Performance and Hardware Concurrency"; Detlef Vollmann; ACCU, April 2015
<https://accu.org/content/conf2015/DetlefVollmann-Atomic%20Counters.pdf>